



ANDERSON SERANGOON JUNIOR COLLEGE
JC2 Preliminary Examination 2026
Higher 2

COMPUTING

Paper 2 (Lab-based)

9569/02

21 August 2026 (Friday)

8-11am

3 hours

Additional Materials: Electronic version of MASTERMIND_GUESSES.txt datafile
 Electronic version of ANIMAL_DATA.json file
 Electronic version of GP_SCORES.txt data file
 Electronic version of ROOMS.txt data file
 Electronic version of STUDENTS.txt data file
 Insert Quick Reference Guide

READ THESE INSTRUCTIONS FIRST

Answer **all** questions.

All tasks must be done in the computer laboratory. You are not allowed to bring in or take out any pieces of work or materials on paper or electronic media or in any other form.

Do NOT write or make any markings on the Insert Quick Reference Guide.

Approved calculators are allowed.

Save each task as it is completed.

The use of built-in functions, where appropriate, is allowed for this paper unless stated otherwise.

Note that up to **6** marks will be awarded for the use of common coding standards for programming style.

The number of marks is given in brackets [] at the end of each question or part question.
The total number of marks for this paper is 100.

This document consists of **15** printed pages and **1** blank page.

Instructions to candidates:

Your program code and output for each of Task 1 to 4 should be saved in a single `.ipynb` file.

For example, your program code and output for Task 1 should be saved as:

`TASK1_<your name>_<centre number>_<index number>.ipynb`

1 Name your Jupyter Notebook as:

`TASK1_<your name>_<centre number>_<index number>.ipynb`

This task is based on Mastermind, a classic code-breaking game. One player (the code-maker) selects a secret sequence of 4 pegs, each coloured Red (**R**), Yellow (**Y**), Green (**G**), or Blue (**B**). Colours may repeat within the code.

The other player (the code-breaker) makes a guess, also a sequence of 4 coloured pegs, and receives feedback in the form of two numbers:

- Hits: the number of pegs that are the correct colour and in the correct position.
- Pseudo-hits: the number of pegs that are the correct colour but in the wrong position.

Each peg in the guess and each peg in the secret code can only be used once when scoring. A peg already counted as a hit cannot also count as a pseudo-hit (this matters when colours repeat).

For instance, if the secret code is **RGBY** and the guess is **GGRR**:

- Position 2 (with G) is a hit.
- Of the remaining letters, only one R can be matched as a pseudo-hit.

The result is thus 1 hit and 1 pseudo-hit.

For each of the sub-tasks, add a comment statement at the beginning of the code using the symbol '#', to indicate the sub-task the program code belongs to, for example:

In [1]:

```
#Task 1.1
Program code
```

Output:

Task 1.1

Write a function `is_valid(code)` that returns a boolean.

The function should:

- check that `code` is a string of exactly 4 characters,
- check that every character in `code` is one of "R", "Y", "G" or "B",
- return `True` if both conditions are met, and `False` otherwise.

Test your function with the following statements:

```
print(is_valid("RYGY"))
print(is_valid("RGB"))
print(is_valid("YYYY"))
print(is_valid("BRGE"))
print(is_valid(""))
print(is_valid(["R", "G", "B", "Y"]))
print(is_valid(6767))
```

Show the output for each test.

[4]

Task 1.2

Write a function `score_guess(secret, guess)`.

It should first use the function in **Task 1.1** to check that both `secret` and `guess` are valid; if either is invalid, return `None`.

Otherwise, the function should:

- count the number of hits,
- count the number of pseudo-hits, and
- return the result as a tuple (`hits`, `pseudo_hits`).

Test your function with the following statements and show the output:

```
print(score_guess("RGBY", "GGRR"))
print(score_guess("RGGB", "RGGB"))
print(score_guess("RGGB", "YYYY"))
```

[5]

Task 1.3**(a)**

Write a function `load_guesses(filename, secret)` that:

- reads a text file which contains eight past guesses, one on each line,
- for each guess, calls `score_guess()` to calculate its hits and pseudo-hits,
- calculates a value for each guess using: $\text{value} = \text{hits} \times 10 + \text{pseudo_hits}$,
- builds a tuple `(slot1, slot2, slot3, slot4, value)` for that guess, where `slot1` to `slot4` are the four individual characters of the guess.

The function should return a list of these tuples.

Call this function with the secret `RGGB` and the given text file `MASTERMIND_GUESSES.txt`.

Output the returned list.

(b)

Write a function `insertion_sort(guesses)` that takes a list of tuples (in the form returned by `load_guesses()`) and sorts them in ascending order of `value`, using the **insertion sort algorithm**.

Call this function using the list from part **(a)**, store the result as `sorted_data`, and print `sorted_data`.

[6]

Task 1.4

Write a **recursive** function `binary_search(data, target)` that searches a sorted list of tuples (in the form returned by `load_guesses()`) for a tuple whose `value` matches the given target.

The function should return the matching tuple, or `None` if no matching tuple is found. If more than one tuple has the same value, returning any one of them is acceptable.

Test your function by searching `sorted_data` for **three** different target values:

21
40
15

Output the result of each search.

[5]

Save your Jupyter Notebook for Task 1.

2 Name your Jupyter Notebook as:

TASK2_<your name>_<centre number>_<index number>.ipynb

PawsHaven Animal Shelter has been rescuing and rehoming dogs, cats and rabbits. The shelter's management has asked you, as a newly recruited developer, to develop a system for their record-keeping. Your job is to load the shelter's existing records into a **NoSQL database** so staff can query and update animal records. You are also to build a queue using a linked list to manage the intake line of animals currently waiting to be processed at the shelter.

You have been given the shelter's current records as a file `ANIMAL_DATA.json`.

For each of the sub-tasks, add a comment statement at the beginning of the code using the symbol '#', to indicate the sub-task the program code belongs to, for example:

In [1]:

```
#Task 2.1
Program code
```

Output:

Task 2.1

Write Python code that:

- creates a **MongoDB** database `paws_haven` and a collection `animals`,
- inserts all records from `ANIMAL_DATA.json` into the `animals` collection,
- queries the collection to count the number of animals currently in shelter and the number of them who have left,
- displays the output in the format:

Animals currently in shelter: _____

Animals who have left: _____

[4]

Task 2.2

Write two MongoDB queries to filter animals based on the following combinations of conditions. For each of them, output the number of matching animals, as well as these basic information for each matching record: `animal_id`, `name`, `species`.

[Turn over

Combination 1:

All vaccinated cats that have left the shelter between 1 March 2025 and 31 January 2026 (both inclusive).

Combination 2:

All unvaccinated dogs and rabbits currently in the shelter with an adoption fee strictly greater than \$45.

[5]

Task 2.3

Animals that are sent to the shelter need to be queued for the staff to process their intake (e.g. health check, paperwork, assigning a kennel).

You will build this queue using a linked list, implemented as `Node` class and a `Queue` class.

- The `Node` class should have two properties:
 - `data`: the value stored in the node,
 - `next`: a reference to the following node in the chain, or `None` if there isn't one.
- The `Queue` class should maintain references to the front and back of the chain of nodes, along with a count of the number of items currently in the queue. It should support the following methods:
 - `enqueue(data)`: creates a new node containing data and adds it to the back of the queue,
 - `dequeue()`: removes and returns the data at the front of the queue, or returns `None` if the queue is empty,
 - `size()`: returns the number of items currently in the queue.

[6]

Task 2.4

Create an `Animal` class to represent an animal at the moment it arrives at the shelter, separate from the full shelter database record used in **Tasks 2.1 – 2.2**.

The class should have the following properties:

Property	Description
<code>animal_id</code>	An ID assigned to the animal at check-in
<code>species</code>	The animal's species
<code>gender</code>	The animal's gender
<code>arrival_date</code>	The date the animal arrived at the shelter, in YYYY-MM-DD format

It should also have a method `display()` to show the animal's information systematically as follows:

Animal ID: _____
Species: _____
Gender: _____
Arrival Date: _____

You will use the following four animals to test your `Queue` and `Animal` classes:

animal_id	species	gender	arrival_date
PH-2001	Dog	Male	2026-08-20
PH-2006	Cat	Female	2026-08-20
PH-2007	Rabbit	Female	2026-08-24
PH-2009	Dog	Male	2026-08-25

Write Python code to:

- first create an empty queue and then enqueue all four animals in the order shown above,
- output the queue's size,
- dequeue each animal in turn, showing that animal's information, until the queue is empty,
- dequeue one more time and print the returned value to show that the queue correctly returns `None` when empty.

[6]

Save your Jupyter Notebook for Task 2.

[Turn over

3 Name your Jupyter Notebook as:

TASK3_<your name>_<centre number>_<index number>.ipynb

A program is required to analyse the cohort's performance on the subject General Paper (GP) performance. It organises class groups by their average GP score, so that best performing and underperforming class groups can be found quickly.

Each class group's data is stored in `GP_SCORES.txt`, one line per class group, in the format:

Class Group: Score1, Score2, Score3, ...

Node	
Attributes	
Data: <code>TUPLE</code>	It stores the class group and its list of scores as a tuple, eg: ("28/01", [27, 39, 39, ...])
left_ptr	It is a reference to the left child, or <code>None</code> if there is no left child
right_ptr	a reference to the right child, or <code>None</code> if there is no right child
Methods	
constructor	Initialises data from the value passed as a parameter left_ptr initialised to <code>None</code> right_ptr initialised to <code>None</code>
average()	returns the average GP score of this node's class group, rounded to 1 decimal place returns 0 if the value list is empty
display_info()	Outputs this node's group and its average GP score in the format : For example: Class group : 28/14 Average GP score : 31.4

GPTree, a Binary Search Tree	
Attributes	
root	a reference to the root node of the tree, or <code>None</code> if the tree is empty

Methods	
constructor	Initialises the <code>root</code> to <code>None</code>
<code>insert(data)</code>	It inserts a new node holding data into the correct position forming a BST based on each class group's average GP score.
<code>inorder()</code>	A recursive method that performs an in-order traversal of the tree and returns a list of tuples in the form <code>(class_group, average_score)</code>. Returns an empty list if the tree is empty.
<code>find_min()</code>	A recursive method to return the node representing the most underperforming class
<code>find_max()</code>	A recursive method to return the node representing the best performing class
<code>depth()</code>	A recursive method to return the depth of the tree using node counting Empty tree = 0 Single node = 1
<code>tree_avg()</code>	A method to return the average GP score for the whole cohort, calculated from every individual score in every class group, rounded to 1 decimal place. Returns 0 if tree is empty or if no scores are stored
<code>classes_below_threshold(threshold)</code>	A method to return a list of tuples in the form <code>(class_group, average_score)</code> for class groups whose average GP score is strictly below <code>threshold</code>. The tuples are returned in pre-order traversal.

For each of the sub-tasks, add a comment statement at the beginning of the code using the symbol '#', to indicate the sub-task the program code belongs to, for example:

In [1]:

```
#Task 3.1
Program code
```

Task 3.1

Write program code to declare the class `Node`, its constructor and its methods. [6]

Task 3.2

Write program code to declare the class `GPTree` and its constructor. [2]

For **Tasks 3.3 to 3.5**, amend the program code written for the `GPTree` class in **Task 3.2**.

Do not create a separate `GPTree` class for each sub-task.

Task 3.3

Amend your program code for the `GPTree` class to include the method `insert(data)` and recursive method `inorder()` as described previously.

You may assume that no two class groups will have the same average GP score, so this method does not need to handle tie-breaking.

Amend your program code:

- to read all records from `GP_SCORES.txt`
- convert each class group record into a tuple containing the class group and a list of integer scores
- pass each tuple to the `insert(data)` method so that all records are inserted into one `GPTree`.

Test your program by calling `inorder()`. For each tuple returned, output one line in the format:

`<class_group> scored an average of <average_score>.`

[8]

Task 3.4

Amend your program code for the `GPTree` class to include the following methods:

- `find_min()`
- `find_max()`
- `depth()`
- `tree_avg()`

The methods `find_min()`, `find_max()` and `depth()` must be implemented recursively.

Using the completed tree, call the appropriate methods and output:

- the class group and the average score of the most underperforming class
- the class group and the average score of the best performing class
- the depth of the tree
- the average GP score of the whole cohort.

Test the program and show all four outputs.

[7]

Task 3.5

Amend the `GPTree` class to include the method `classes_below_threshold(threshold)`.

The method must:

- traverse the BST in **pre-order** sequence
- select class groups whose average GP score is strictly below `threshold`
- use the ordering of the BST so that it does not visit a subtree that cannot contain an average below `threshold`
- return a list of tuples in the form `(class_group, average_score)`

The tuples must appear in the order in which the qualifying nodes are encountered during the **pre-order traversal**.

Test the method using the following test data:

1. 25.0
2. 27.0

For each test, output:

- the threshold score
- each tuple in the returned list, one per line
- the number of tuples in the returned list

Show the two sets of results separately.

[5]

Save your Jupyter notebook.

4 Name your Jupyter Notebook as:

TASK4_<your name>_<centre number>_<index number>.ipynb

A group of 30 students from ASRJC are embarking on a Technology Immersion Trip to Shenzhen, China. To help teachers manage the trip, a room allocation system named ShenZhenGo is to be built. The system stores student and room allocation data in a relational database and makes it searchable through a web application.

Students are allocated to Twin (2-person) or Triple (3-person) rooms which are single-gender specific across three floors of the hotel.

Each room is identified by a room number whose hundreds digit indicates the floor — for example, room 209 is on Floor 2.

A small number of students hold leadership roles as Student Leaders.

The database will have 2 different tables to store data on:

1. **Student**
2. **Room**

The fields in each table are:

Student

- `Student_ID` — a unique identification number assigned to each student
- `Student_Name` — full name of the student as registered in the student identification records
- `Student_Class` — class the student belongs to. Example: 28/01, 28/27
- `Student_Gender` — gender of the student. Stored as **M** for Male student and **F** as Female student
- `Student_Leader` — Indicate **1** if the student is a Student Leader, **0** if not
- `Room_ID` — identification of the room the student is assigned to. Each student is assigned to exactly one room.

Room:

- `Room_ID` — unique room identification number. The hundreds digit indicates the floor. Example, 201 is on Floor 2
- `Room_Type` — type of room accommodation. Can only be Twin (2 persons sharing) or Triple (3 persons sharing)

For each of the sub-tasks 4.1 to 4.3, add a comment statement at the beginning of the code using the symbol '#', to indicate the sub-task the program code belongs to, for example:

In [1]:

```
#Task 4.1  
Program code
```

Output:

Task 4.1

The data files `ROOMS.txt` and `STUDENTS.txt` each store comma-separated values for the corresponding table. The first row of each file contains the header of the respective columns.

In the `STUDENTS.txt`, `Student_Leader` is stored as the text `TRUE` or `FALSE`.

Write a Python program that uses SQL code to:

- create the database `SHENZHENGO.db` with the two tables given, setting appropriate data types for the table fields, and defining primary and foreign keys for each table,
- read in the data from each file and store each record in the appropriate table,
- display the following information for every student: `Student_Name`, `Student_Gender`, `Student_Leader`, `Room_ID`, sorted in ascending alphabetical order of `Student_Name`

Write a Python program that uses SQL code to display the following information for every student:

- `Student_Name`
- `Student_Gender`
- `Student_Leader`
- `Room_ID`

[8]

Task 4.2

Write a Python program that uses SQL code to find the number of distinct rooms that have at least one student assigned.

Run the query and show its output in the Jupyter Notebook.

[2]

Task 4.3

Each room should contain no more than one Student Leader.

Write a Python program that uses SQL to identify rooms with **more than one Student Leader**. For each such room, display the `Room_ID` and `Student_Name` of all Student Leaders assigned to that room.

Sort the results in ascending order of `Room_ID`. For students assigned to the same room, sort the results in ascending alphabetical order of `Student_Name`.

Run the query and show its output in the Jupyter Notebook.

[4]

Save the completed Jupyter Notebook containing **Tasks 4.1 to 4.3**.

Task 4.4

A web-based platform should allow teachers to view the room allocation of every student and reassign a student to a different room.

Write a Python program and the necessary files to create a Flask web application that uses `SHENZHENGO.db`.

You may assume that the reassignment:

- does not exceed the capacity of the destination room
- does not create a mixed-gender room
- does not result in a room containing more than one Student Leader

You are not required to validate these conditions.

The application must use exactly two HTML template files.

Update interface – `index.html`

The Update interface must contain:

- a **form** receiving a student's name and new `Room_ID`
- a **button** for submitting the form
- an area **for displaying a confirmation message** after the update
- a **navigation link** to the Display interface.

When the form is **submitted**, the application must update the student's `Room_ID` in the database and display a suitable confirmation message.

Display interface – `display.html`

The Display interface must contain a table with the following four columns:

- Student Name
- Student Gender
- Student Role (displays 'Leader' or 'Member')
- Room ID

The table must:

- include a header for every column
- display every student
- be sorted by `Room_ID` in ascending order

- reflect the most recent assignment
- use black borders
- appear on a page with background colour **#98FF98**

The page must also contain a **navigation link** back to the **Update** interface.

Save your Python program as

Task4_4_<your name>_<centre number>_<index number>.py

with any additional files or subfolders as needed in a folder named

Task4_4_<your name>_<centre number>_<index number>

Run the web application and test with the following reassignment:

Student Name: Ng Jun Hao Room ID: 311
--

Save the following webpage outputs after performing the reassignment:

1. Save the **Update** interface, showing the confirmation message.

Task4_4_update_<your name>_<centre number>_<index number>.html

2. Save the **Display** interface, confirming that **Ng Jun Hao** is shown in room **311**.

Task4_4_display_<your name>_<centre number>_<index number>.html

[11]

BLANK PAGE